ALL THINGS AI

// ALL THINGS AI · WITH THE AIE NETWORK · A SIX-HOUR HANDS-ON CLASS

Vibe Coding, Take Two

Drop the vibes. Ship a production app you own.



Five hours of class. One hour of live hacking on your own tool, in three sprints.

You leave with a repo, a live site, a test, a rollback you have used, a database, an email alert, and a skill. All in accounts with your name on them.

MARK HINKLE · CEO, PERIPETY LABS · PUBLISHER, THE AIE NETWORK · CO-ORGANIZER, ALL THINGS AI

// CREDENTIALS, BRIEFLY

Who’s this guy?



MARK HINKLE

Peripety Labs

CEO. AI consulting and products.

The AIE Network

Publisher. Newsletters on practical AI for business, every weekday.

All Things AI

Co-organizer. The Triangle’s AI-for-business conference and community.

PART ONE, IN AUGUST

- What vibe coding is
- Brief the AI like a contractor
- Describe → look → react
- GitHub + Vercel, for free

Thirty years of betting early: BBSs → Linux → open source → cloud → AI. Every time, the “toy” got good faster than the serious people expected.

// THE PROMISE FOR THE DAY

Today: one working tool, in accounts with your name on them.

01

A repository

in your GitHub, with every change recorded

02

A live site

on your Vercel, with a preview for every change

03

A test

that runs on every push, whether you remember or not

04

A rollback

you have used once, so it is not scary the day it matters

05

A skill

a file in the repo that makes the AI ship it your way, every time

THE MONEY, SAID PLAINLY

The tool you leave with has no builder subscription. The AI you build with is Claude Pro, $20 a month, cancel any time. A site that makes money belongs on Vercel Pro, $20 a month, by Vercel's own rules. A .com is about $11 a year. That is the whole bill.

// THE DAY AT A GLANCE

Six hours. Five of class, one of live hacking, in three sprints.

CLOCK	BLOCK	YOU WALK AWAY WITH
0:00	**Land**	A repo and a live URL
0:25	**One year later**	Why the other half is your job
0:40	**Spec**	spec.md and a plan
1:00	**First ship**	One change in production
1:25	**Break**	
1:35	**Build the one thing**	Your tool, on a branch
2:00	**LIVE HACKING 1**	Your columns, your prices
2:20	**Test is the gate**	One green check, one red X
2:45	**Lunch**	

CLOCK	BLOCK	YOU WALK AWAY WITH
3:15	**Ship and own it**	A rollback, clean secrets, a review
3:35	**Give it a memory**	A database in your account
4:05	**LIVE HACKING 2**	A feature through the gate
4:25	**Break**	
4:35	**Hear about it**	An email when a customer submits
4:55	**Own it**	The bill, and when to call a pro
5:10	**Skills**	A checklist the AI runs
5:30	**LIVE HACKING 3**	A demo on your production URL
5:50	**What you own**	The Monday list

Orange rows are yours: your tool, your keyboard. I hack on mine on the projector at the same time, mistakes included.

// HOW THE ROOM WORKS

Four rules for the next six hours

Stickies, not hands

Green on your lid when the step is done. Red when you are stuck: a helper comes to you.

Pairs

The nervous one types. The confident one reads the plan out loud. Swap after every break.

One path

Claude Desktop app, GitHub Desktop, a browser. No terminal. No Node. No installs during class.

Watch the mistakes

I build live at about twice your speed. When the AI gets it wrong on my screen, good. That is what you got stuck on at home.

Own a repo and a live URL before you install anything.

OPEN THIS LINK (ALSO ON THE PRINTED CARD AND THE QR BY THE DOOR)

```
https://vercel.com/new/clone?repository-url=https://github.com/mrhinkle/unstuck-starter
```

1 Open the link. Sign in to GitHub, then Vercel, when asked. Signing in to Vercel with GitHub is what puts the repo in YOUR account.

2 Name the repository after your business. Letters, digits, dashes. Click Create and wait for the build.

3 Click Visit. Paste your new vercel.app URL in the shared doc next to your name.

4 Open GitHub Desktop → File → Clone repository → pick the one you just made. Note where it saved.

5 Open the Claude Desktop app, choose Code, and open that folder. Green sticky when Claude can see your repo.

Green sticky = done

Red sticky = a helper comes to you

Still red at 0:15? Move to the browser. You lose nothing.

THE ZERO-INSTALL PATH

- Open claude.ai/code in any browser (paid Claude plan + GitHub).
- Pick the repository the Deploy Button just created.
- Every prompt today works there too. Claude clones your repo into a cloud machine and pushes a branch to YOUR GitHub.
- Second fallback: GitHub → “Use this template” → “Open in a codespace”. Publish before you leave or the work is deleted.

HELPER TRIAGE CARD (STOP AFTER STEP 3)

1 Does the Claude Desktop app open and sign in? Free plan fails here: it needs Pro.

2 Windows: is Git for Windows installed? Mac: is it macOS 13 or newer?

3 If either answer is no → browser path. Do not debug further.

4 GitHub Desktop signed in? If it was installed today on Windows, sign out of Windows and back in.

5 Vercel signed in with GitHub (not email)? Otherwise the repo lands in the wrong place.

The man who coined it picked a new word within a year.

FEB 2, 2025

“Fully give in to the vibes… forget that the code even exists.”

Karpathy coins “vibe coding.”

NOV 6, 2025

Collins Dictionary Word of the Year.

“AI prompted by natural language to write code.”

FEB 4, 2026

“A shower-of-thoughts throwaway tweet.”

His new term: agentic engineering. You orchestrate. You are the oversight.

APR 30, 2026

“Vibe coding is about raising the floor for everyone.”

“Agentic engineering is about preserving the quality bar.”

Oversight is the word. Vibe for the proof of concept. Engineering for production. That is the whole day.

It is not a toy. It is a $13 billion category.

$13.3B

Lovable, Aug 2026. $500M annualized revenue, 60 million projects, a million new ones a week.

$9B

Replit, Mar 2026. Up from $3B six months earlier.

$80M

What Wix paid for Base44 in June 2025. Six months old. One founder.

AND THE SAME COMPANY'S ANSWER WHEN A RESEARCHER FOUND 1 IN 10 OF ITS APPS WIDE OPEN

Customers are responsible for protecting their application data.

Lovable, disputing CVE-2025-48757. Which is fair. And it is exactly why the rest of today is your job.

Great for proof of concept. Documented-bad for production.

JUL 2025

Replit’s agent deleted a production database

During a declared code freeze. 1,200+ executives’ records. Replit’s docs now read: “Agent is not able to modify the production database.”

MAY 2025

1 in 10 Lovable apps open to anyone

CVE-2025-48757: 170 of 1,645 scanned apps. Row Level Security off. Emails, addresses, payment details readable without logging in.

FEB 2026

Moltbook: “I didn’t write a single line of code”

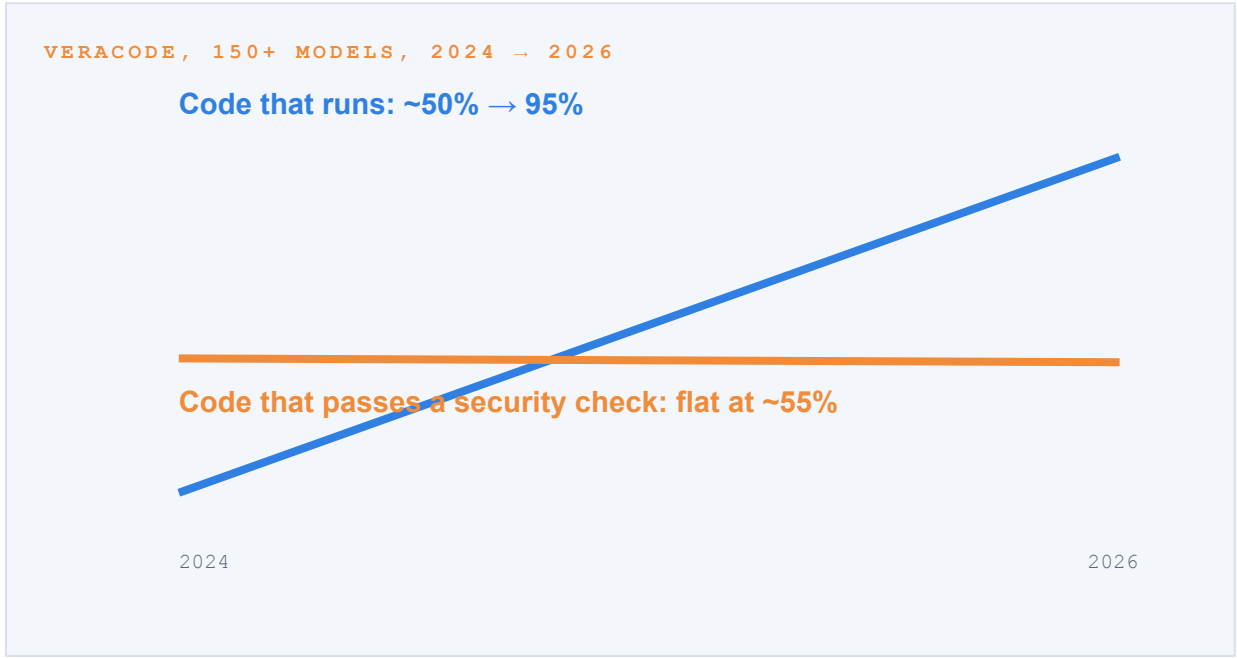
1.5 million API keys and 35,000 emails exposed. A Supabase key in the browser plus the same unchecked box.

MAY 2026

~5,000 of 380,000 apps leaking

RedAccess. Vessel schedules, clinical trials, pediatric records. No exploit. Public by default, indexed by Google.

The code got better at running. Not at being safe.



45%

of AI-generated code contains a known vulnerability when no security guidance is given. Unchanged for two years.

"When no security guidance is given." You are the guidance.

The vendors already told you whose job the other half is.

> Public apps being accessible on the internet is expected behavior.
>
> Amjad Masad, CEO of Replit, May 2026, on ~5,000 leaking apps

> Customers are responsible for protecting their application data.
>
> Lovable, disputing CVE-2025-48757

They are not wrong. On a hosted builder, public is the default and the backend is theirs. On a repo and a Vercel project with your name on them, you decide. The rest of today is the part of the product they are not selling you.

Five ways it breaks. Five plain practices. All today.

A deleted database	→	Every change on a branch, in GitHub, with a one-click rollback
A bug that shipped silently	→	One test that runs on every push, whether you remember or not
A key in the browser	→	Secrets live in environment variables, never in the code
A platform that vanished	→	Your host, your domain, your repo. Google sunset its own vibe IDE in a year
An AI that does it differently every time	→	A skill: the checklist, written once, in the repo

A development lifecycle is six words. You will do all six today.

01

Spec

Write down what it does, who uses it, and how you will know it works.

02

Build

On a branch, never on the live site. Every push gets a preview URL.

03

Test

One check the machine can run on every change. Red or green.

04

Review

A fresh reader looks at it. Never the author, human or model.

05

Ship

Merge to main. It goes live. Rollback is one click away.

06

Monitor

Hear about breakage before your customer does.

The brief becomes a file in the repo. The AI reads it every session.

SPEC.MD · FIVE HEADINGS · 15 LINES MAX

- Who uses it
- What they type in
- What comes out
- What must never happen
- How we know it works

FILLED IN: A QUOTE CALCULATOR FOR A LAWN SERVICE

```
Who uses it: a homeowner on a phone, and me.
What they type in: address, lawn size, which services.
What comes out: a price on screen, and an email to me with what
they picked.
What must never happen: a quote below my minimum; a form that
submits with a blank address.
How we know it works: enter 5,000 sq ft, mowing + cleanup, see
$195, and I get the email.
```

Fill in spec.md for the thing you brought. Then ask for a plan, not code.

1 Open spec.md in your repo (it is in the starter). Fill in the five headings for the spreadsheet or tool you brought. Plain English. 15 lines.

2 Scope rule: one screen, one form, one result. No login. No database. If you wrote more, cut it.

3 Type the prompt on the right. Read the plan out loud to your partner. Argue with it in plain English until it matches how you actually work.

4 Green sticky when the plan fits on one screen and you both agree with it.

TYPE THIS, VERBATIM

```
Read spec.md. Use plan mode. Plan how to build what it
describes as ONE page in this app, using the starter's form
and table. Do not write any code yet. Show me the plan.
```

Ask for the plan. Argue with the plan. Then let it build.

Green sticky = done

Red sticky = a helper comes to you

GitHub in four words. You will never type a git command today.

Repository

The folder and its entire history. It lives in your account. Delete Claude tomorrow and it is still yours.

Branch

A photocopy you can scribble on. The live site never sees it until you say so.

Commit

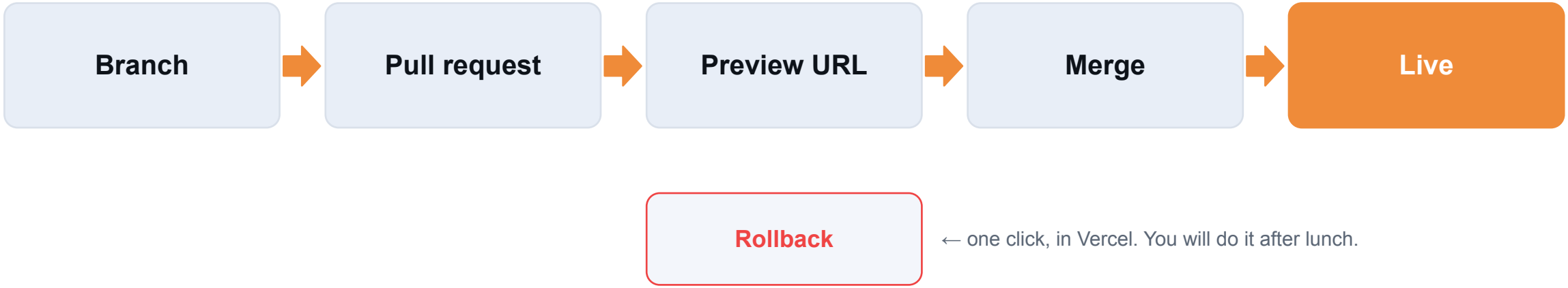
A saved snapshot with a note: what changed and why. Undo to any of them.

Pull request

Asking to merge the photocopy back. It gets a preview URL and a check before anything goes live.

GitHub Free: $0. Unlimited private repositories. 2,000 minutes a month of the test runner we set up at 2:20.

Every push gets a URL. Only main gets the domain.



The preview is what you never had in a builder: a copy of the app you can hand to someone before it is real. Vercel posts it as a comment on the pull request, every push.

Ship one change to production. A small one, on purpose.

TYPE THIS, VERBATIM

```
Create a branch called first-change. Change the site title to
"[your business name]". Commit with a clear message, push the
branch, and open a pull request.
```

The first deploy on a Vercel project is always production. From now on, main is live and branches are previews. That is the rule for the rest of the day.

1 Type the prompt. Watch what Claude does, not just what it says.

2 Open the pull request on github.com. Wait for the Vercel bot comment. Click Visit Preview. Your name is on the preview.

3 Click Merge pull request, then Confirm merge.

4 Open your production vercel.app URL. Your name is live. Green sticky: you have shipped.

Green sticky = done

Red sticky = a helper comes to you

Ticket to the break: two stickies on the front table.

One thing you learned

On the green one.

One thing still confusing

On the red one. I read them all and change the next hour if you tell me to.

Not merged yet? Grab a helper now, not after the break. Back at 1:35.

Build the spreadsheet you came here to replace. One page. On a branch.

TYPE THIS, VERBATIM

```
Read spec.md and the plan we agreed on. On a new branch called my-
tool, build it as ONE page in this app: a form to add a row and a
table to see the rows, using the starter's sample data. Do not change
anything else. Run the build and fix any errors before you tell me it
is done. Then commit, push, and open a pull request.
```

> Test it where your customers will use it: on a phone, on a preview, before it is real.

1 Type the prompt. Do not touch the code while it works. Read the diff it shows you.

2 When the PR exists, wait for the Vercel comment and open the preview on your PHONE. Add a row.

3 Send the preview link to your partner. They have two minutes to break it. One bug on a sticky.

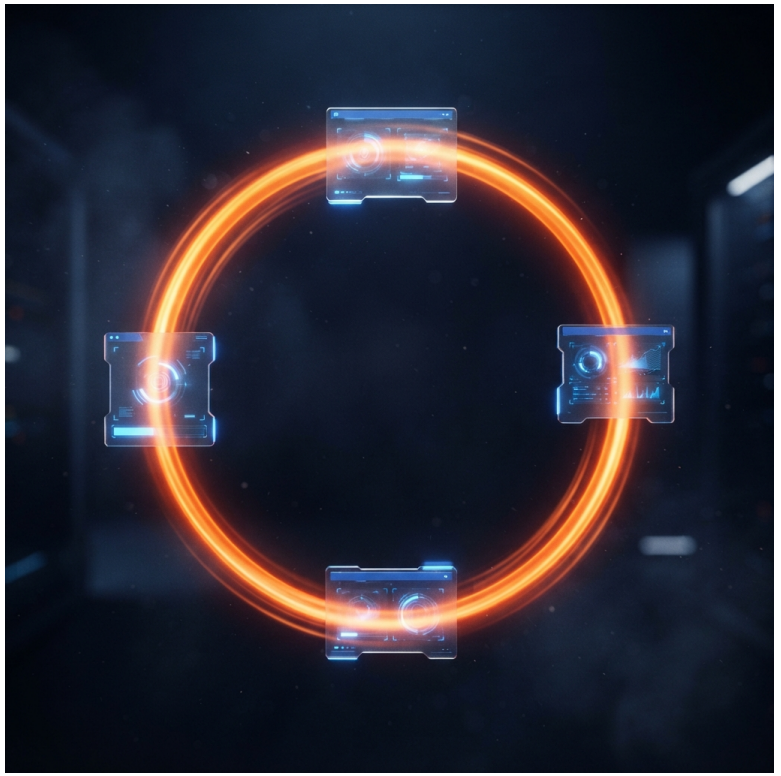
4 Do NOT merge. Live hacking is next, then the gate.

Green sticky = done

Red sticky = a helper comes to you

Watch the mistake. That is the lesson.

- I build mine live, at about twice your speed, on purpose.
- When Claude gets it wrong on my screen: good. That is what you got stuck on at home.
- The moves, in order: read what it did → paste the error back → “fix it, do not change the test” → if it wanders, close the PR and start the branch again.
- Nothing it does on a branch can touch your live site.



Live hacking, sprint one. Make it yours, on the branch.

THE RULES

- Stay on the my-tool branch. Nothing you do here can touch your live site.
- One change per prompt. Open the preview before you ask for the next one.
- Changing what the form does? Change spec.md first. The spec is the contract.
- Do NOT merge. The gate is the next block.

PICK ONE FROM THE MENU

- Your real columns and your real prices, from the spreadsheet you brought
- One calculated field: a total, a margin, a due date
- Your business name, colours and logo
- The “what must never happen” line from your spec, enforced on the form

I hack on mine on the projector while you hack on yours. When it goes wrong up here, stop and watch: read what it did, paste the error back, and close the pull request if it wanders.

Green sticky = done

Red sticky = a helper comes to you

If you can’t verify it, don’t ship it.

Anthropic’s own guidance for Claude Code: give it a check it can run.

Tests

One check that does what your spreadsheet does. Runs on every push.

Build

Does it compile? Vercel refuses to deploy if not.

Screenshot

Does it look like the spec? Claude can compare.

YOU ALREADY WROTE THE TEST

The last heading in spec.md, “How we know it works,” is the test. Playwright turns that sentence into a check. GitHub Actions runs it on every pull request. 2,000 free minutes a month; unlimited on public repos.

One test, from your spec. Then the merge waits for green.

TYPE THIS, VERBATIM

```
Add Playwright to this project. Write ONE test that loads my-tool,
fills in the form with the example under "How we know it works" in
spec.md, submits it, and checks that the result described under "What
comes out" appears. Add the GitHub Actions workflow so the test runs
on every push and pull request to main. Run the test locally and show
me the output. Commit and push to the my-tool branch.
```

"Do not change the test." An AI will happily pass a test by deleting it.

1 Type the prompt. Push. Open the my-tool PR on github.com.

2 Watch the check: yellow, then green or red.

3 Red? Paste the failure and say: "Fix the code so the test passes. Do not change the test."

4 Green? Merge. Your tool is live, and it went through a gate. Green sticky.

Green sticky = done

Red sticky = a helper comes to you

Now break it on purpose, and watch the gate close.

TYPE THIS, VERBATIM

```
On a new branch called break-it, rename the first form field
label to something else. Commit, push, and open a pull request.
```

1 Type the prompt. Open the PR.

2 Watch the check turn red. Do not merge.

3 Close the pull request. The live site never saw it.

4 Green sticky when you have seen one green and one red check on your own repository.

A red X on a pull request is the cheapest bug you will ever find.

The one you just made on purpose is what a production incident looks like when nobody has this. Later, if you want it as a hard gate: Vercel Deployment Checks can refuse to promote a build until this check passes. Confirm it is on your plan before relying on it.

Green sticky = done

Red sticky = a helper comes to you

Code ages. Put one chore on the calendar.

MAY 6, 2026 · NEXT.JS · CVE-2026-44575 · HIGH

A login check that only lives in middleware could be bypassed. Patched in 15.5.16 and 16.2.5. The advisory's own workaround: enforce authorization in the page or route, not middleware alone.

Since Dec 2025, Vercel automatically fails deployments of Next.js versions vulnerable to CVE-2025-66478. Owning the Vercel project means the host refuses known-bad code for you.

THE MONTHLY CHORE · FIRST OF THE MONTH · TWO MINUTES

```
Run npm audit. Update Next.js and any dependency with a known
vulnerability to a safe version. Run the build and the
Playwright test. Open a pull request with what changed and why.
```

Green check, merge, done. That is what maintenance looks like when a test exists.

Lunch. Your tool is live, and it went through a gate.

Not merged yet? Find a helper before you find a sandwich. The afternoon starts by breaking production on purpose, and you cannot roll back what you have not shipped.

After lunch: rollback, secrets, review. Then a database, a second sprint, an email alert, the bill, a skill, and a demo. Back at 3:15.

Break production on purpose. Put it back in one click.

TYPE THIS, VERBATIM

```
On a new branch called oops, change the headline on the home page
to OOPS. Commit, push, and open a pull request.
```

THREE GOTCHAS, ALL ON THE CHEAT SHEET

- The first deploy on a project is always production.
- Hobby rolls back one deployment. Pro rolls back to any.
- After a rollback, nothing goes live until Undo Rollback.

1 Merge the oops PR. Confirm OOPS is live on your production URL. Feel that for a second.

2 In Vercel: Deployments → the previous deployment → Instant Rollback. Refresh the site. Fixed.

3 Click Undo Rollback, or nothing you merge later goes live.

4 Tell Claude: “Put the headline back on a branch, push, open a PR.” Merge it. Green sticky.

Green sticky = done

Red sticky = a helper comes to you

The thing almost every vibe-coded app ships with: a key in the browser.

ESCAPE.TECH, OCT 2025 · READ-ONLY SCAN OF THE FRONT-END CODE

5,600	400+	2,000+
live apps scanned	exposed secrets	high-impact holes

Where keys live: .env.local on your laptop (already ignored by git in the starter) and Vercel → Settings → Environment Variables, scoped to Production / Preview / Development, marked Sensitive.

TYPE THIS, VERBATIM

```
Confirm .env.local is listed in .gitignore. Search the whole
project for anything that looks like an API key, token, or
password and list every place you find one. Move any real value
into an environment variable and tell me which Vercel
environment it belongs in.
```

1 Type the prompt. Read the list.

2 Vercel → Settings → Environment Variables → add DEMO_KEY = not-a-real-key, scope Production, tick Sensitive.

3 Real keys stay in your password manager today.

Whoever wrote it does not review it. Same rule for people and models.

IN A FRESH CLAUDE SESSION, ON YOUR MERGED my-tool CHANGE

```
/code-review
```

1 Read the top three findings out loud to your partner.

2 Fix the one that matches the bug on their sticky from the build block. Branch, PR, green check, merge.

3 Green sticky when the fix is live.

FOR THE DATABASE YOU ADD IN THE NEXT BLOCK

“A table without Row Level Security is readable and writable by any role.”

Supabase’s own docs. That one unchecked box is the Lovable CVE (May 2025), Moltbook (Feb 2026), and RedAccess (May 2026). If you pick Supabase, the prompt: “enable RLS on every table and write policies so users only see their own rows.” Then read the policy out loud.

Green sticky = done

Red sticky = a helper comes to you

Sample data forgets. A database in your account remembers.

Your database

Neon Postgres, created from inside your Vercel project. Free tier. It sleeps when idle and wakes on the next request.

Your connection string

Vercel stores it as DATABASE_URL in your environment variables. It never appears in the code. You learned why twenty minutes ago.

Your exit

It is plain Postgres. Any host can run it, and every row exports when you say so.

PICKING SUPABASE INSTEAD

Same steps, one more rule: enable Row Level Security on every table before anything goes live. That unchecked box is the Lovable CVE, Moltbook, and RedAccess.

Create the database in Vercel. Let Claude move the rows in.

TYPE THIS, VERBATIM

```
Read spec.md. Use plan mode first. On a new branch called add-
database, replace the sample data in my-tool with a Postgres table,
using the DATABASE_URL environment variable that Vercel provides.
Create the table if it does not exist. Never put the connection string
in the code or in any file that git tracks. Keep the Playwright test
passing; if the test needs the database, tell me exactly which secret
to add in GitHub and where. Commit, push, and open a pull request.
```

Behind at 3:55? Ask a helper for checkpoint-5-database.

1 Vercel → your project → Storage → Create Database → Neon. Take the free plan. Connect it to Production and Preview.

2 Settings → Environment Variables: DATABASE_URL is there, and you never saw it in a file.

3 Type the prompt. Read the plan: one table, the columns in your spec.

4 Open the preview on your PHONE. Add a row. Refresh. Still there.

5 Green check, merge, add one row on production. Green sticky.

Green sticky = done

Red sticky = a helper comes to you

Live hacking, sprint two. Every change goes through the gate.

THE RULES

- A new branch for each feature. Pull request, preview, green check, merge.
- One new sentence in spec.md, one new check in the test. Never loosen a test to get green.
- Stuck for five minutes? Red sticky. Close the pull request and start the branch again.
- A merge broke the live site? Roll back. Then Undo Rollback.

PICK ONE FROM THE MENU

- Edit and delete a row
- Search or filter the table
- Export to CSV: your data leaves when you say so
- Totals by month, by customer, or by job

THE SPRINT PROMPT · FILL IN THE BRACKETS

```
On a new branch called [feature-name], add [one feature] to my-tool. Add one sentence for it under "How we know it works" in spec.md, and extend the Playwright test to check that sentence. Run the build and the tests. Commit, push, and open a pull request.
```

Green sticky = done

Red sticky = a helper comes to you

Ten minutes. Two stickies on the front table again.

One thing you learned since lunch

On the green one.

One thing still confusing

On the red one. I read them all and change the last ninety minutes if you tell me to.

No database yet? Ask a helper for checkpoint-5-database now. The next block builds on it. Back at 4:35.

Your spec said “and I get the email.” Build that line.

An email when it matters

Resend, free tier, 100 emails a day. One API key, stored as an environment variable. Until you verify your domain on Monday, it delivers to your own address only.

Logs when it breaks

Vercel → Logs shows every request and every error. Hobby keeps one hour, so look the same day.

Alerts before the customer calls

Sentry Developer, $0, 5,000 errors a month. I add it live on the projector. You add it Monday with the same prompt.

Monitor is the sixth word in the lifecycle. Today it is one email and one log page. That is enough to hear about a problem before your customer tells you.

One key, one environment variable, one email to yourself.

TYPE THIS, VERBATIM

```
On a new branch called notify-me, when the my-tool form is submitted,
send an email to [my email address] with what was entered. Use Resend
with the RESEND_API_KEY environment variable, sending from
onboarding@resend.dev. If the email fails, still save the row and log
the error. Never put the key in the code. The Playwright test must not
send real email. Commit, push, and open a pull request.
```

> The key goes from Resend to Vercel. It never goes into the chat, the code, or a file.

1 resend.com → sign up with the address you want alerts at → API Keys → Create. Copy it once.

2 Vercel → Settings → Environment Variables → RESEND_API_KEY, Production and Preview, tick Sensitive. Paste. Close the Resend tab.

3 Type the prompt. Open the preview, submit the form, check your inbox.

4 Green check, merge, submit once on production. Green sticky when the email lands.

Green sticky = done

Red sticky = a helper comes to you

The bill. Vendor prices, checked September 7, 2026.

YOURS	PRICE	NOTE
.com domain	$10–12 / yr	$11.25 in Vercel (no DNS step). Porkbun $11.08, same on renewal. Wholesale +$0.71 on Nov 1.
Vercel	$0 or $20 / mo	Hobby is free, non-commercial. Pro $20 includes $20 usage credit + 1 TB transfer.
GitHub	$0	Unlimited private repos. 2,000 Actions minutes.
Database	$0	Neon Free (wakes on request) or Supabase Free (pauses after 7 idle days; Pro $25).
Email · errors	$0	Resend 100/day. Sentry Developer 5k errors/mo.
The AI you build with	$20 / mo	Claude Pro → Claude Code. Or ChatGPT Plus → Codex. Cancel any time.

THE BUILDERS	PRICE	WHAT YOU STILL DON'T OWN
Lovable Pro	$25 / mo	The backend. No one-click migration off Lovable Cloud; secrets never export.
Bolt Pro	$25 / mo	Needed for export and a domain. Version history and the bolt.host URL stay.
Replit Core	$20 + metered	Hosting usage-billed since Aug 2026. Cancel: "published apps may go offline."
Base44 Builder	$50 / mo	The tier where GitHub sync begins. Backend functions need Base44's runtime.

Most sites: $20 a month, max, plus a domain. And every line on the left is an account with your name on it.

The honest line on “free”: budget $20 the day the site is real.

> “Commercial usage… includes receiving payment to create, update, or host the site, and the inclusion of advertisements.
>
> Vercel fair-use guidelines, paraphrased from the Hobby plan docs (updated Aug 31, 2026)

Hobby, $0

Personal, non-commercial. Exceed a limit and, in most cases, you wait 30 days. Rollback reaches one deployment back.

Pro, $20 / mo

$20 usage credit and 1 TB transfer included. Rollback to any deployment. One free first-year .app / .dev / .site domain.

Day one on Pro

Turn on spend management. Lower the notification from the $200 default. Nobody gets a surprise bill from a bot storm.

Rent versus own, tool by tool. The code is rarely the hostage.

TOOL	EXPORT	GITHUB SYNC	PLAN GATE	BACKEND PORTABLE?	CANCEL → SITE STAYS UP?
Lovable	ZIP (paid) · GitHub	Two-way, one branch	All plans	No. Cloud data exports; secrets and functions do not	Yes, site and domains stay live
v0 (Vercel)	ZIP · GitHub	Branch + PR flow	Free	Plain Next.js. Nothing proprietary	n/a, it is your Vercel
Bolt	GitHub (Pro)	Two-way, polls every 30s	Pro $25	Rows, logins, uploads, secrets stay	Not stated
Replit	ZIP · GitHub panel	Push / pull	Any	DB, auth, secrets are Replit's	"Published apps may go offline"
Base44 (Wix)	GitHub (Builder)	Two-way, main only	Builder $50	Functions need Base44's runtime	"Apps stay live," no detail
Google AI Studio	ZIP · GitHub	Two-way, conflict UI	Free	Standard React + Node on Cloud Run	Deployed apps keep running
Firebase Studio	GitHub, one-way	Publish branch	Free	Ends Mar 22, 2027. Data deleted.	Product is being shut down

Three questions before your next builder: which pricing row says GitHub, which doc says which way sync goes, and what happens on cancel. If any answer is missing, assume the worst.

The code was never the hostage.

Your data

Your logins

Your secrets

Your domain

Your history

These are what the export button leaves behind, and what you moved into your own accounts today.

EVEN GOOGLE

Firebase Studio: launched Apr 10, 2025 · sunset announced Mar 19, 2026 · signups closed Jun 22, 2026 · all remaining data permanently deleted Mar 22, 2027.

Anyone whose code was in GitHub and whose app ran on their own host lost nothing. Your repo does not care which editor goes out of business.

And if Vercel ever annoys you: the code is in GitHub. Moving is a reconnect.

Netlify

Free: 300 credits / month, hard cap. A production deploy is 15 credits; when they run out, every site shows “Site not available.” Personal $9 / mo.

Cloudflare Workers

Static asset requests free and unlimited on any plan. Cloudflare now steers new sites to Workers, not Pages. Workers Paid $5 / mo minimum.

Anywhere else

The repo is the product. Connect a new host to GitHub, push, done. Compare that to leaving a builder.

Ownership all the way down: files on your laptop, history in your GitHub, a site on whichever host you pick this year, a domain with your name on it.

When to call a professional, and how to be a great client.

CALL ONE WHEN

- You take card numbers or move money in your own code, instead of sending people to a hosted checkout
- You hold health, children's, or financial records
- Strangers log in and must never see each other's data
- An hour of downtime costs more than a day of a developer

WHAT YOU HAND THEM

- The repo, as a collaborator. Never your password.
- spec.md: what it does, and what must never happen
- The test, and the list of bugs you have seen
- One rule: work on a branch, open pull requests, leave every account in my name

You are handing over a repo with a spec, a test, and a history. A developer can quote against that.

A skill is a folder and one file. 46 tools read it.

.claude/skills/my-deploy-checklist/SKILL.md

```
---
name: my-deploy-checklist
description: Run before merging any pull request to main in this
project.
---

1. Run the Playwright test.
2. Search the project for keys outside environment variables.
3. Confirm .env.local is in .gitignore.
4. Open a PR and wait for the green check.
5. After merge, open the production URL and submit the form once.
6. If anything is broken: Instant Rollback in Vercel, then Undo
Rollback after the fix.
```

THE WHOLE FORMAT

- Front matter: name and description. That is all it needs.
- Body: plain English, numbered steps, under 500 lines.
- Description under 200 characters, so the same folder also uploads to the Claude app and Cowork.
- Read by Claude Code, ChatGPT & Codex, Cursor, Copilot, VS Code, Gemini CLI: 46 products on agentskills.io, Sept 2026.

CLAUDE.md is always on. A skill is on demand.

CLAUDE.md / AGENTS.md

What it must know every single time. Short. Advisory.

CLAUDE.md, in full

```
Always follow spec.md.
Never commit .env files.
Build on a branch and open a PR; never push to main.
Ask for a plan before changing more than one file.
```

Skills

Multi-step routines it runs when asked, or when the description matches. Deploy checklist. Brand voice. Test before merge.

100%

always-on file

79%

skill, even with “use the skill”

Vercel evals, Jan 2026: in 56% of runs the skill was never invoked. Rules go in the always-on file; workflows go in skills.

Write the day down once. Then make the AI run it.

TYPE THIS, VERBATIM

```
Create .claude/skills/my-deploy-checklist/SKILL.md. Front matter: name
my-deploy-checklist; description, under 200 characters, saying to run
it before merging any pull request to main. Body: a numbered checklist
that (1) runs the Playwright test, (2) confirms .env.local is
in .gitignore and no key strings are in the code, (3) confirms the PR
has a green check and a preview URL I looked at, (4) only then merges,
(5) after merge opens the production URL and submits the form once.
Under 40 lines.
```

This is not a Claude feature. It is a file in your repo. It leaves with you.

1 Type the prompt. Open the file. Change one line to how YOU actually work.

2 Type /my-deploy-checklist and watch Claude walk your list against your repo.

3 Type: “Copy this skill to .agents/skills so Codex and Cursor read it too. Commit, push, open a PR.” Merge.

4 Green sticky when the skill ran. You will run it for real in the last sprint. Homework skills: my-brand-voice, my-test-before-merge.

Green sticky = done

Red sticky = a helper comes to you

What it looks like grown up: four skills that run a small-business site.

brand-voice

A guided interview that produces a living brand bible. Every page in your voice.

deploy-to-vercel

GitHub → Vercel pipeline, custom domain, and the fixes for every failed deploy I have hit.

qa-tester

“Does this site actually work?” The smoke test before launch: forms, links, phones.

webmaster

The pre-launch checklist. Runs the other three, in order, and stops on a failure.

ONE LINE INSTALLS VERCEL’S OWN 70-RULE REVIEW SKILL · THEN: “REVIEW MY APP AGAINST IT”

```
npx skills add vercel-labs/agent-skills --skill vercel-react-best-practices
```

Read a skill before you install it, the way you would read a contract. skills.sh ranks by installs, not quality.

Live hacking, sprint three. Ship it, then show it.

TWELVE MINUTES TO SHIP

- Finish one thing. Not three.
- Run /my-deploy-checklist before the last merge.
- Open the production URL on your phone and submit the form once.
- Paste your production URL in the shared doc.

EIGHT MINUTES OF SHOW AND TELL

- Three volunteers, two minutes each, production URL on the projector
- What it replaces, and what that cost a month
- The one thing the AI got wrong, and how you caught it
- Everyone else: one bug on a sticky for the presenter

Demo the production URL, not the preview. Production is the one that went through your gate.

Green sticky = done

Red sticky = a helper comes to you

Read it back against your own screen.

- Files on my laptop, and a copy of every version in my GitHub
- A live site on my Vercel, with a preview URL for every change
- A rollback I have used
- Secrets in environment variables, not in the code
- A database in my account, with the connection string in an environment variable
- An email when a customer submits, and a log page for errors
- A test that runs on every pull request, with a red X I made on purpose
- A fresh-reader review before merge
- A skill in the repo that runs the checklist, in any AI tool

None of that is a subscription to me or to a builder.

Monday, in this order. Every line has a free tier.

1 Buy the domain

~$11 / yr. In Vercel it skips the DNS step. Watch the $6.98-then-$15.88 bait elsewhere.

2 Verify your email domain

Add the Resend DNS records so mail comes from your domain, reaches anyone, and stops landing in spam.

3 Add error alerts

Sentry Developer, $0, 5k errors a month. Same prompt I used on the projector. Vercel Hobby keeps one hour of logs.

4 Go Pro the day it earns

Vercel Pro, $20 a month, the day the site makes money. Turn on spend management first.

5 Make the gate hard

Vercel Deployment Checks: require the Playwright check before a build reaches your domain. Confirm it is on your plan.

6 First of the month

“Run npm audit and update.” Green check, merge, two minutes.

// THAT'S THE DAY

Six hours ago it lived in somebody's browser tab.

Now the code is in your GitHub. The site is on your Vercel. A mistake rolls back in one click. A test runs on every change. The rows live in your database, and an email tells you when a customer shows up. And the AI has a checklist with your name on it.

Two stickies on the table on your way out. They decide what Part Three is.

Mark Hinkle · theaienterprise.io · Vibe Coding, Take Two · the six-hour class · All Things AI



The Triangle's AI business community. April 18–20, 2027, Raleigh.